Fattorizzazione come moltiplicazione “reversed”

Davide Camino

November 11, 2026

Abstract

Alcuni esperimenti su come fare la fattorizzazione di un numero scrivendo un programma che fa moltiplicazioni tra ineri. L'idea è di implementare in Verilog un programma che moltiplica due valori A e B ottenendo come risultato C. Si trasforma questo programma in un circuito digitale e successivamente in un problema Ising. Una volta ottenuto il problema Ising si sfrutta la reversibilità offerta dal modello per fissare il risultato C della moltiplicazione e ottenere i valori A e B.

Nella sezione 1 si introduce il programma Verilog discutendone le caratteristiche, in 2 si presenta l'esperimento di fattorizzazione con i relativi risultati, in 3 si studia la dimensione del problema Ising in funzione del numero da fattorizzare, infine in 4 si discutono i limiti dello studio e si propongono spunti futuri.

1 Programma Verilog

Il programma Verilog che implementa la moltiplicazione tra due valori può essere specificato a differenti livelli di asptrazione. Possiamo costruire un programma che operi bitwise e spechifichi esattamente le operazioni applicate a ciascun bit dei fattori al fine di ottenere il prodotto oppure possiamo sfruttare gli operatori di “alto livello” offerti da Verilog e assegnare direttamente a C il prodotto di A e B. In questo caso come primo esperimento è stata seguita quest'ultima strada essendo solo un'esplorazione preliminare. Il codice “template” per questo circuito è mostrato nel Listato 1, la moltiplicazione è definita alla linea 7.

```
1  module Multiplier (
2          input  wire [2:0] A,
3          input  wire [2:0] B,
4          output wire [5:0] Mul
5          );
6
7          assign Mul = A * B;
8
9  endmodule
```

Listing 1: Multiplier in Verilog

Altro dettaglio importante è il numero di bit che assegnamo ad ogni variabile (linee da 2 a 4), utilizzando Verilog abbiamo un controllo puntuale del numero di bit utilizzati da ogni variabile, qeusto è molto utile per tenere sotto controllo la dimensione del problema QUBO finale e studiare come cambia al varirare delle dimensioni dei dati in ingresso e uscita. Per dimensionare l'output del circuito si è tenuto in considerazione che:

$$C = A \cdot B,$$

di conseguenza:

$$\log_2 C = \log_2(A \cdot B) = \log_2 A + \log_2 B. \tag{1}$$

Questo rende molto semplice definire la dimensione di C conoscendo le dimensioni di A e B, anche se presenta dei limiti che discuteremo in seguito.

2 Esperimento

Per testare la fattorizzazione si è scelto di fissare il valore di C a 35, questo è il valore più alto che: si possa scrivere su 6 bit, sia il prodotto di due numeri primi e non sia un quadrato perfetto. Fissato il valore di C ci aspettiamo che la coppia $\langle A, B\rangle$ valga $\langle 5, 7\rangle$ oppure $\langle 7, 5\rangle$.

2.1 Set Sperimentale

Il problema in forma Ising è stato ottenuto con la pipeline presentata nel report scorso[**?**]. Per sintetizzare i circuiti logici sono stati usati diversi set di porte logiche ottenendo circuiti con diverso numero di gates e problemi Ising con diverso numero di qbit:

1. { `AND`, `NOT` }: 68 gates, 68 qbits;
2. { `AND`, `OR`, `NOT` }: 61 gates, 61 qbits;
3. { `AND`, `OR`, `NAND`, `NOR`, `NOT` }: 38 gates, 38 qbits;
4. { `AND`, `OR`, `NAND`, `NOR`, `XOR`, `NOT` }: 30 gates, 38 qbits (per ogni porta `XOR` è necessario un qbit ancilla);

Per ottenere il problema Ising ad ogni porta è stata assegnata ad ongi porta una funzione pseudobooleana che usasse meno ancille possibili[1] perchè si è visto che la qualità dei risultati migliora più sono ridotte le dimensioni del problema, cioè meno qubit si usano[**?**].

I problemi in forma Ising sono stati risolti con il metodo del Simulated Annealing (SA) e del Simulated Quantum Annealing(SQA), entrambi i metodi sono offerti dal framework Ocean di D-Wave[2]. Per ottenere dei risultati statisticamente rilevanti ogni problema è stato risolto 30 volte per ogni metodo raccogliendo per ogni run $2^{16} \approx 65{\cdot}000$ samples. Ogni sample rappresenta un assegnamento di A e B. Terminate le 30 run per ogni diverso assegnamento di A e B è stata clacolata la media e la deviazione standard del numero di osservazioni di quell'assegnamento.

In questo caso non sono stati fatti esperimenti con QAOA perché la simulazione dell'algoritmo avrebbe richiesto troppa memoria. Per risolvere il problema Ising si è usata la libreria OpenQUOPS[3] che sfrutta Qiskit e Qiskit Aer per descrivere e simulare il circuito. Attualmente il metodo di simulazione utilizzato da `AerSimulator` è `statevector` che ha un costo spaziale $O(2^n)$ dove n è il numero di qubits. Dato questo costo non è possibile eseguire la simulazione con un numero così elevato di qubits.

[1] La definizione della funzione pseudobooleana, sotto forma di matrice triangolare superiore è siponibile a URL
[2] URL
[3] URL

2.2 Risultati

I risultati ottenuti sono presentati nella Figura 2 . Dall'immagine si nota che il SA produce dei risultati di qualità migliore rispetto al SQA, ma entrambi esibiscono più o meno lo stesso comportamento, le distribuzioni degli assgnamenti hanno circa la stessa forma, anche se il SQA presenta soluzioni più rumorose. Questo suggerisce che entrambi i metodi rimangano "incastrati" negli stessi minimi locali, anche se al SQA succede più frequentemente che al SA.

Possiamo anche notare che la qualità del risultato aumenta, a prescinere dal metodo di risoluzione utilizzato, man mano che il numero di qbits diminuisce. Confrontando le soluzioni dei circuiti ottenuti con i set di porte 3 e 4 nell'elenco numerato 2.1 vediamo che a parità di qubits il circuito con più gates si comporta meglio. Questo comportamentoi merità un'esplorazione più approfondita, per il momento possiamo ipotizzare che più gates generino un circuito (clalssico) più profondo e di conseguenza una matrice più sparsa. Una forma di energia di questo tipo probabilmente è più facile da minimizzare.

3 Analisi della dimensione del QUBO

Per studiare come aumenta la dimensione del problema in funzione della dimensione dei fattori bisogna prima studiare la dimensione del circuito sintetizzato. In questa prima analisi non studiamo il circuito dal punto di vista dell'ingegneria elettronica, ci limitiamo invece a osservare quanto abbiamo prodotto attraverso Yosys.

In particolare a partire dal template del Listato 1 e usando la Formula 1 per dimensionare il risultato generiamo via via circuiti che accenttano in input numeri sempre più grandi. Nell'Immagine 1 si vede come aumenta il numero di gates in funzione del numero di bit dedicati a ciascun fattore della moltiplicazione. Il valore di R^2 molto vicino a 1 suggerisce che la crescita quadratica ipotizzata descriva bene l'aumentare del numero di gates del circuito.

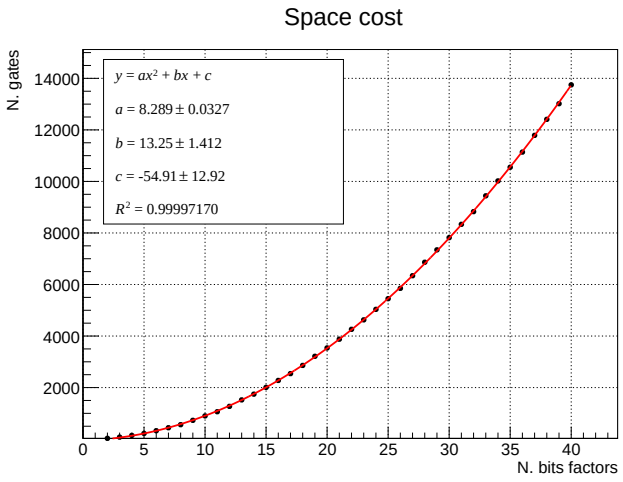


Figure 1: Gates in funzione dei bits dei fattori A e B

Dato che per ogni gate fa aumentare di un valore costante il numero di qubits necessari a rappresentare il problema Ising possiamo concludere che anche la dimensione del problema finale aumenta quadraticamente rispetto alle dimensioni dell'input.

Questo è un risultato importante perchè se la dimensione del circuito crescesse esponenzialmente rispetto alla dimensione dell'input passeremmo da un problema che si risolve classicamente in tempo esponenziale a un problema che richiede spazio esponenziale, di fatto mettendoci in una situazione peggiore.

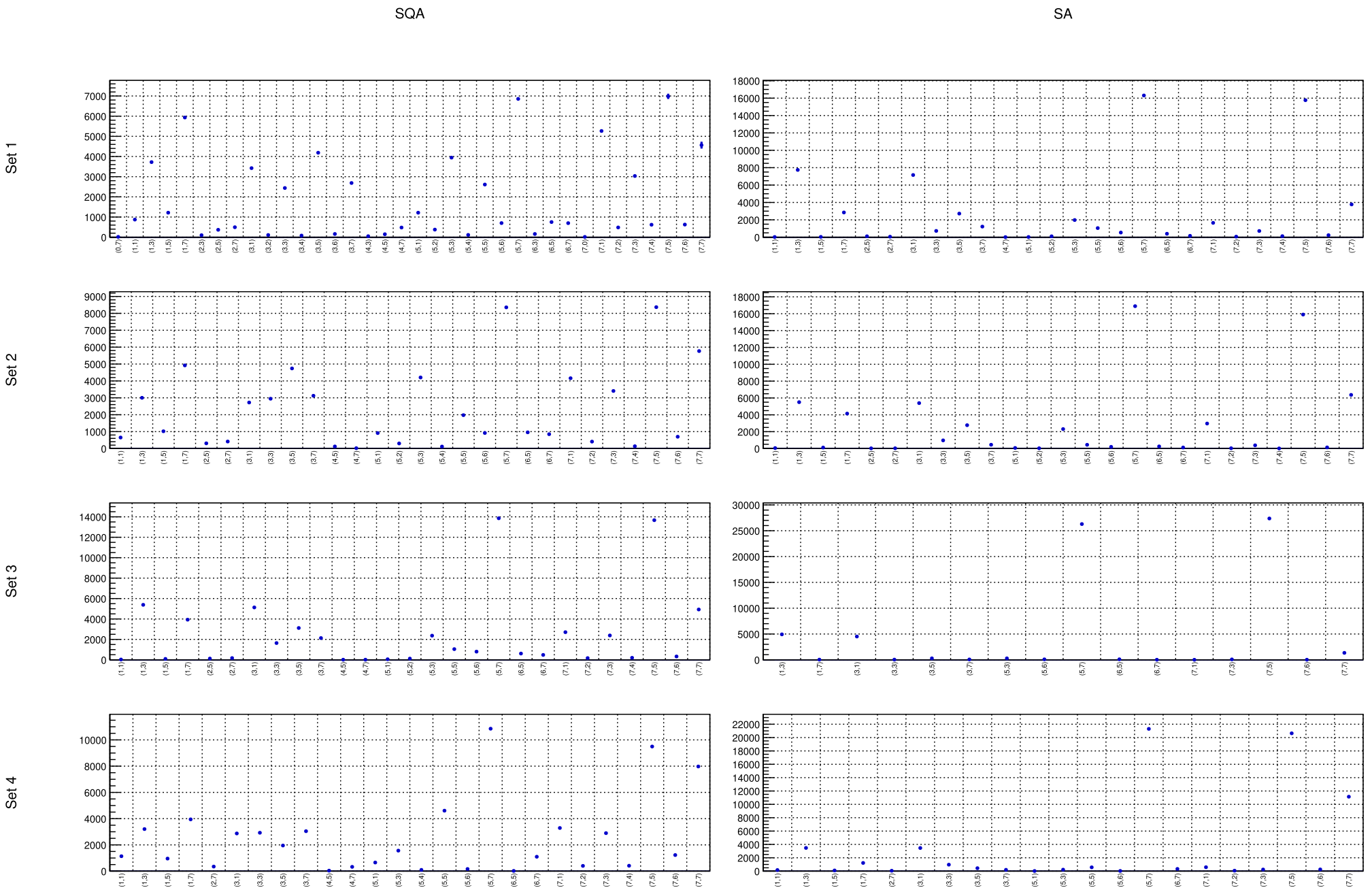


Figure 2: La Figura si legge come una tabella, sulle righe ci sono i diversi set di porte utilizzate, sulle colonne i diversi metodi di risoluzione; sull'asse x di ogni grafico leggiamo gli assegnamenti di A e B osservati; sull'asse y il numero medio di osservazioni per un certo assegnamento, l'errore associato alla misura è la deviazione standard.

3.1 E Shor?

Da quello che ho trovato l'aricolo più recente è del 2024 [**?**] e detto N il valore da fattorizzare, con $n = \log_2 N$ il numero di qubit richiesti per implementare l'aloritmo di Shor è $\frac{n}{2} + o(n)$ con un numero di gates che è dell'ordine di $O(n^3)$ per una profondità di circuito che è $n^2 \log^3 n$ e va eseguito più volte (l'articolo stima che per fattorizzare una chiave RSA a 2048 bit servano 1730 qubit logici e il circuito debba essere eseguito 40 volte).

Questo risultato è interessante, infatti nonostante il numero di qubit logici richiesto dall'algoritmo di Shor sia infariore, il circuito quantistico da costruire cresce più rapidamente di quanto aumentino le dimensioni del problema Ising, questo significa anche più errori introdotti e un numero maggiore di qubit fisici per compensare la poca affidabilità dei gates quantistici. Questo non significa che questo approccio sia più efficiente dell'algoritmo di Shor, ma vuol anche dire che non va scartato aprescindere solo perchè il numero di qubits chresce quadraticamente.

4 Limitazioni e spunti

È necessario studiare un po' di elettronica per capoire come descrivere un moltiplicatore, Yosys fa un ottimo lavoro, ma quando il numero di gates aumenta molto l'ottimizzazione inizia a richiedere veramente molto tempo.

QAOA non testato si possono usare altri metodi oltre a `statevector`

La dimensione dell'output va bene se si parte dall'input, al contrario non funziona più molto bene ade esempio...